ECGR 4101/5101 - LAB 8: TIC TAC TOE GAME

Date	Author	Group No
12-03-2024	Poojitha Rajapuram Indhuja Gudluru	G09

Table of Contents

1 Overview of this Document 1
2 Project Overview 1
3 Requirements 2
4 Bill of Materials (BOM) 4
5 Hardware architecture 5
6 Software architecture 6
7 Test Plan 10
8 Calculations 16
9 Setbacks and their solutions 17
10 Conclusion 17
11 References 18

1 Overview of this Document

This document offers an in-depth look at the development of the Tic Tac Toe game, which was created using two MSP430 microcontrollers. The game provides an interactive experience for two players, where they communicate via UART and see their actions reflected on LCD screens. The report covers various aspects of the project, including the project overview, the design and architecture of both hardware and software, detailed requirements, the bill of materials, and the test plan. Additionally, it highlights the key calculations, challenges encountered during the project, and the solutions that were implemented to overcome those challenges.

2 Project Overview

The goal of this project is to design and create a two-player Tic Tac Toe game using MSP430 microcontrollers. Each player will have their own microcontroller, with the game state displayed on an LCD screen. The system will use UART communication to keep the game synchronized in real-time between the two players. To improve the user experience, the design will include additional peripherals, such as buttons for making moves, LEDs to show whose turn it is, and a buzzer to provide sound effects for actions like placing a marker on the grid. The work will be divided into hardware and software development:

Hardware Development: Designing the circuit and selecting the components.
Software Development: Writing the firmware to handle user input, game logic, communication, and display updates.
A demonstration video of the gameplay can be viewed at the following link:
[https://youtube.com/shorts/tPNtdjPzVBY?si=FrMUOBZy8Pe3qNJv].

3 Requirements

Microcontroller Requirements

- **Req_MC001:** Each player module should have one MSP430G2533 microcontroller.
- **Req_MC002:** The microcontroller must be in a 20-pin DIP package.
- **Req_MC003:** The microcontroller should control the game logic.
- **Req_MC004:** The microcontroller should control the sequence of player turns.
- **Req_MC005:** The microcontroller should validate proper movement between player modules.

Power Supply Requirements

- **Req_Pow001:** Each player module must have its own independent power source (battery).
- **Req_Pow002:** The system must utilize a voltage regulator to ensure a consistent 3.3V output for each MSP430G2533 microcontroller.
- **Req_Pow003:** The power supply must sustain a 3.3V output with a tolerance of ±5% under all game operating conditions.
- **Req_Pow004:** The power supply should provide sufficient current to operate both MSP430G2533 microcontrollers and all connected peripherals, including the LCD, LEDs, and buzzer.

User Interface Requirements

- **Req_UI001:** The system must have an LCD to display the Tic Tac Toe grid game.
- **Req_UI002:** The LCD should display messages such as “Start,” “Game Over,” “Player A’s turn”, “Player B’s turn”, “Player A Wins,” and “Player B Wins.”
- **Req_UI003:** The LCD must indicate the current player’s turn.
- **Req_UI004:** Each player should have a button to navigate across the Tic Tac Toe grid.
- **Req_UI005:** Each player should have a button to place their selected mark (X or O) on the chosen grid position.

Audio and Visual Requirements

- **Req_AV001:** Each player module must include an LED.
- **Req_AV002:** The LED must blink to indicate the player’s turn.
- **Req_AV003:** Each player module must include a buzzer to signal when a player places their move.
- **Req_AV004:** The buzzer must emit a distinct sound each time the player navigates to a new grid position.
- **Req_AV005:** The buzzer must emit a different, distinct sound when the player places their mark.
- **Req_AV006:** The LCD display for the opposing player must show only the final placement of the other player’s mark (X or O) without displaying the intermediate navigation across the grid.

Game Operation Requirements

- **Req_Game001:** The game must allow either of the players to start a new game once the previous game is completed.

- **Req_Game002:** The system must allow each player to choose whether they want to represent “X” or “O” at the start of the game.
- **Req_Game003:** During gameplay, the system must ensure that the player’s move is visible only on their own board and not directly visible to the opposing player.

Game Communication Requirements

- **Req_Comm001:** The two player modules must communicate through a cabled connection.
- **Req_Comm002:** The communication cable should be at least 3 feet in length.
- **Req_Comm003:** The communication link should be transmitted and receive data without interference.

Hardware Layout Requirements

- **Req_HW001:** All components should be assembled on a custom PCB (Printed Circuit Board).
- **Req_HW002:** The PCB must not use any jumper wires.
- **Req_HW003:** Headers should be included to enable in-circuit programming of the microcontroller.

Testing Requirements

- **Req_Test001:** Each button should be tested individually to ensure accurate functionality and responsiveness.
- **Req_Test002:** The LED should blink exclusively during the respective player’s turn.
- **Req_Test003:** The buzzer must sound only upon confirmation of movement.
- **Req_Test004:** The I2C communication link among the two microcontrollers and the LCD should be tested to ensure proper data communication and accurate display functionality.
- **Req_Test005:** The game logic must be tested to ensure proper handling of player turns, move validation, win conditions, and draw scenarios.

Operational Reliability Requirements

- **Req_Relia001:** The system must ensure all button inputs are processed without any lag.
- **Req_Relia002:** The LCD must refresh the displayed board’s state immediately after a move is placed.

Programming and Code Requirements

- **Req_Prog001:** The program must be written entirely in the C programming language.
- **Req_Prog002:** The program should not use inbuilt delay cycles for timing adjustments.
- **Req_Prog003:** The program must be structured, formatted, and readable based on general coding guidelines.
- **Req_Prog004:** The program must use functions wherever possible to ensure modularity and reusability.

Project Design and Assembly Constraints

- **Req_Const001:** The team must adhere to the deadlines for Billing of Materials, schematic, test plan, and demo.
- **Req_Const002:** The design must use MSP430G2533 microcontrollers only.
- **Req_Const003:** The assembly should use a soldered PCB.

.

4 Bill of Materials (BOM)

The following Bill of Materials (BOM) outlines all the components required to build the Tic Tac Toe game system. This includes the microcontrollers, input/output devices, power supplies, and other peripherals needed for the hardware setup. Each component has been carefully chosen to ensure optimal performance and compatibility for the project.

Item No	Component Name	Quantity	Description	Manufacturer/Vendor	Unit Price($)	Total price($)
1	MSP430G2533 Microcontroller	2	16-bit, 20-pin DIP package, low-power MCU	Texas Instruments/Digikey	11.99	23.98
2	SSD1306 OLED Display	2	128x64, I2C Interface, 3.3V compatible	Samsung/Amazon	2.99	5.98
3	Buttons	4	Push buttons	School Lab	0.00	0.00
4	LEDs	2	Light Emitting Diodes (LEDs) for player turn indication and move acknowledgment.	School Lab	0.00	0.00
5	Buzzer	2	buzzer for audible feedback on game moves.	Local Electronics Shop	0.79	1.58
6	Resistors	10	Resistors for I2C Pull-Up	School Lab	0.00	0.00
7	Voltage Regulator	2	3.3V Regulator	Local Electronics Shop	1.66	3.32
8	9V Rechargeable Battery	2	Power Supply for Each Module	Amazon	3.95	7.9
9	Battery Clip	2	To Connect battery to an electrical circuit	School Lab	0.00	0.00
10	I2C Communication Cable	1	3-ft, 4-wire cable for I2C communication	Local Electronics Shop	3.93	3.93
11	Headers	9	Male/Female Headers for easy connections	Local Electronics Shop	0.99	7.92
12	Perf Board	2	Perf Board for Final Assembly	Amazon	1.99	3.98
13	20 Pin dip IC socket	2	Socket for 20-pin DIP microcontroller	School Lab	0.00	0.00
				Total Budget of the project	**58.59**	

5 Hardware architecture

- The hardware architecture for the Tic Tac Toe game is built around several key components to ensure smooth functionality and interaction between players. These components include:
- **MSP430G2553 Microcontrollers:**
  Each player is assigned their own microcontroller, which handles the game logic and coordinates all interactions with the hardware components. The microcontroller is responsible for interpreting button presses, updating the LCD display to reflect the current game state, controlling the LEDs to indicate whose turn it is, managing the buzzer for sound effects, and ensuring seamless UART communication between the two gameboards. This setup allows for smooth gameplay and real-time updates during the game.

- **LCD (SSD1306):**
  The 128x64 OLED SSD1306 LCD display is used to show the 3x3 Tic Tac Toe grid, giving players a clear view of the current game state. It updates in real-time to reflect player moves and indicates the game's outcome, whether it's a win, loss, or tie. The display communicates with the MSP430 microcontroller through the I2C protocol, allowing for efficient data transfer and smooth updates to the screen. This makes it easy for players to track their progress and the game's result.

- **Buttons:**
  The buttons are used for two primary actions: navigating the game marker across the grid and placing the marker (either 'X' or 'O') in the selected position. Each player has a set of buttons corresponding to these actions.

- **LEDs:**
  LEDs are placed on the board to indicate which player's turn it is. This visual indicator helps the players keep track of the game's progress.

- **Buzzer:**
  The buzzer is used for audio feedback during key game actions, such as placing a marker or completing the game. It provides an auditory cue when the game reaches critical stages or if an invalid move is made.

- **UART Communication:**
  UART (Universal Asynchronous Receiver/Transmitter) communication is used to facilitate real-time data exchange between the two MSP430 microcontrollers. This allows the game state and player moves to be synchronized between the two boards, ensuring consistency and interaction between players.

- **Power Supply:**
  The system is powered by a battery, with a voltage regulator ensuring a steady 3.3V supply to the microcontrollers and all connected peripherals. This regulator is crucial for maintaining consistent voltage, which helps keep the hardware components running smoothly and reliably throughout the game. By stabilizing the power, it ensures that

everything from the microcontroller to the display and buttons functions properly, even as the battery's charge fluctuates.

- **Voltage Regulator:**
  The voltage regulator ensures that the power supplied to the microcontroller and other components is stable and meets the required specifications, preventing any malfunctions due to power fluctuations.

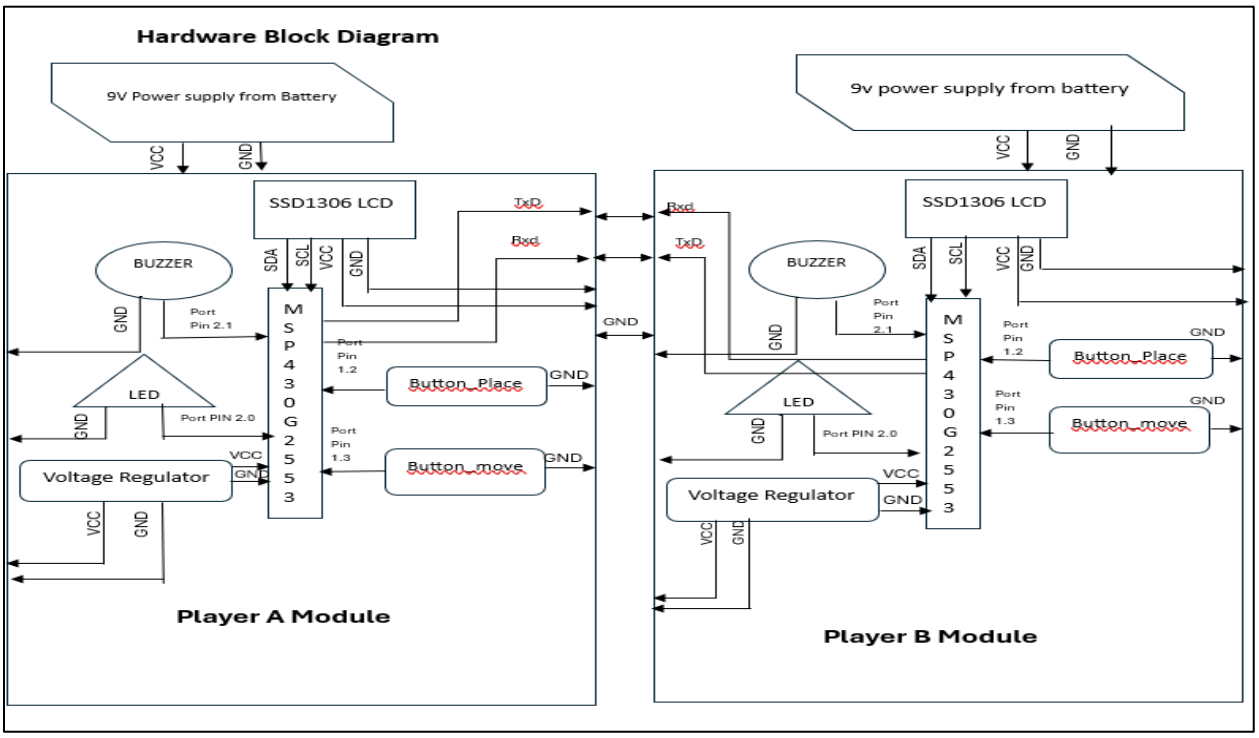


Fig: 1.1

The hardware block diagram (Figure 1.1) above illustrates the design of the Tic Tac Toe game built with MSP430 microcontrollers. For this project, the PCB (Printed Circuit Board) needs to be developed to integrate all the necessary components, including the MSP430G2553 microcontroller, LCD display, buttons, LEDs, a buzzer, and a communication interface.

6 Software architecture

The software architecture for the Tic Tac Toe game implemented in the lab is centered around modularity and communication between two microcontrollers. It follows a structured design with distinct layers for hardware initialization, game logic, and communication. The architecture utilizes UART for inter-board communication to synchronize game states, I2C for controlling the SSD1306 OLED display, and GPIO for button inputs, LEDs, and the buzzer. The game logic is modularized to handle different states such as marker selection, gameplay, checking game status,

and reset, which are managed using a state machine approach. This ensures a clear separation of responsibilities, enabling easier debugging and scalability. The use of interrupt-driven UART and timer-based debouncing enhances responsiveness, while the I2C interface efficiently updates the display to reflect real-time game progress. The architecture emphasizes reliability, modularity, and user-friendly interactions, aligning with the hardware constraints of the MSP430 microcontrollers.

6.1 Software architecture modules

The software architecture is divided into the following modules:

- Int main();
- void initI2C();
- void initUART();
- void initButtons();
- void initBuzzer();
- void initLED();
- void initDebounceTimer();
- void LED_inidcation();
- void buzzer_emit_sound();
- void initial_message_display_LED();
- void Marker_selection_display();
- void drawGrid();
- void drawMarker();
- void navigating_marker();
- void marker_placement();
- void i2c_write();
- void Data_transmission_UART();
- void Processing_Rx_data();
- void Timer_for_debouncing();
- void Game_status_monitor();
- void resetGame();

6.2 State machine Diagram

This below is the state machine diagram representing the different stages and transitions in the Tic Tac Toe game.

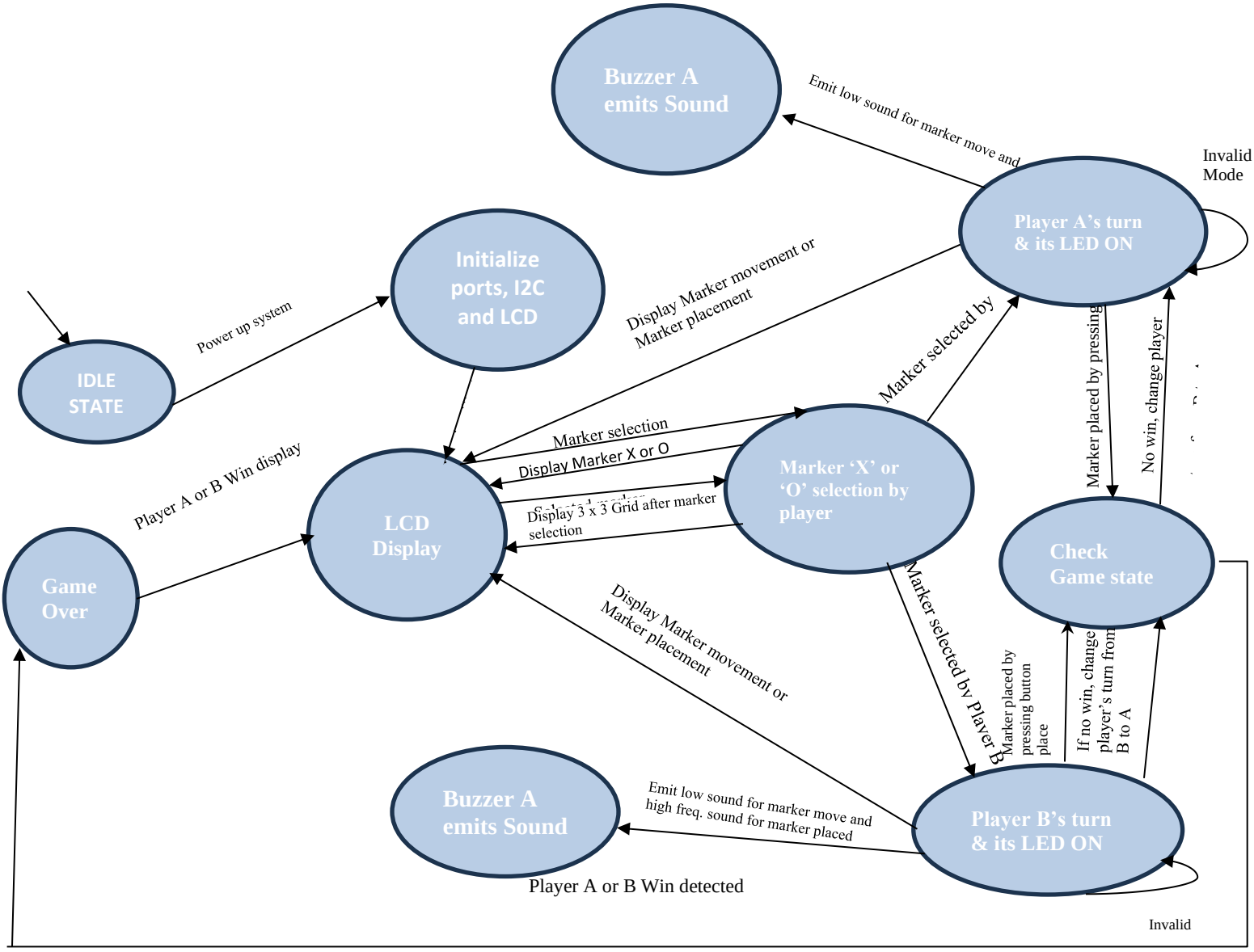


Fig : 1.2

6.3 Algorithm of the software

//Algorithm for Lab 8: Tic Tac Toe Game

// Authors: Poojitha Rajapuram, Indhuja Gudluru, Date: 11/21/2024

// Version 1.1

Initialization:

- Initialize hardware components: I2C, UART, Buttons, Buzzer, LEDs, and Timer.
- Initialize SSD1306 display for game grid.
- Set global variables (markerX, markerY, grid, existingPlayer, gameOver, etc.).

Main Loop:

- Continuously check if the game is over.
    - If the game is over, reset the game.
    - If the game stage is 1 (Gameplay), draw the grid and wait for player actions.
- Monitor for incoming UART data and process it when received.

Button Handling:

- If Button 1 (P1.4) is pressed:
    - Select the 'X' marker for Player 1.
    - Transition to gameplay stage.
    - Send UART signal to other board indicating Player 1's choice.
    - Indicate Player 1’s turn with LED.
- If Button 2 (P1.3) is pressed:
    - Select the 'O' marker for Player 2.
    - Transition to gameplay stage.
    - Send UART signal to other board indicating Player 2's choice.
    - Indicate Player 2’s turn with LED.

Marker Navigation:

- In gameplay stage, allow the player to navigate the marker across the grid by pressing buttons.
- Update the position of the marker on the display.

Marker Placement:

- When Player presses a button for placing a marker:
    - Check if the position is empty.
    - Place the current player's marker ('X' or 'O') on the grid.
    - Send UART data to the other board indicating the marker position.

- Check the game status (win, draw, or continue).

Game Status Monitoring:

- Check for a win condition: three markers in a row, column, or diagonal.
- If a player wins, declare the winner and send a win signal via UART.
- If the grid is full and no winner is found, declare the game as a draw.

Game Reset:

- Once a win or draw is detected, prompt the game to reset.
- Clear the grid and reset all variables.
- Display a reset message on the LCD and indicate the end of the game.
- Send UART reset signal to the other board.

Display Updates:

- Continuously update the LCD display to show the current grid and player markers.
- Use I2C for communication between the microcontrollers and the display.

UART Communication:

- Send and receive game data (player choices, marker positions, game over signals) using UART.
- Handle UART receives and transmits interrupts.

7 Test Plan

7.1 Test Environment

- **Microcontroller:** MSP430G2533 (20-pin DIP)
- **Peripherals:** LCD, LEDs, Buttons, Buzzer
- **Power Supply:** Battery-powered with a voltage regulator (3.3V ± 5%)
- **Communication Link:** UART over a 3-ft cable
- **Software:** Written in C, modular design

7.2 Purpose

The purpose of this test plan is to:

- **Find and Resolve Errors:** To ensure that all components, hardware and software, work without defects by systematically finding and fixing issues during the testing phase.
- **Verify Functional Requirements:** The system shall follow the specifications of grid display, turn alternation, mark placement, invalid move handling, and game outcomes.
- **Performance Stability:** Make sure the system works and is stable within a variety of gameplay conditions, power fluctuations, and rapid input scenarios.
- **Test User Experience:** Test the responsiveness of user inputs, the feedback of LEDs and buzzers, and clarity of the LCD display to ensure a smooth and intuitive gaming experience.

7.3 Test Cases

Functional Testing

Test case no	Test Case	Input	Expected Output	Actual Output	Pass/Fail
F01	Game Initialization	Power on both player modules	LCD on both modules displays "Start."	Displayed "Start."	Pass
F02	Indicating the player turn	Start game.	Active player's LED blinks, inactive player's LED remains off.	LED blinked only for active player.	Pass
F03	Navigation Button Functionality	Press navigation button once (binary: 0b01 for right, 0b10 for down).	LCD cursor moves to the adjacent cell based on direction input.	Cursor moved correctly.	Pass
F04	Placement Button Functionality	Press placement button on Player A's or B's module.	Player's mark (X/O) should appear in the selected grid cell.	Marker is placed accurately.	Pass

F05	Turn Switching	Player A/B places a mark.	Turn switches to Player B/A; their LED starts blinking.	Turns alternate worked correctly.	Pass
F06	Win Detection	Complete a row (e.g., (1,1), (1,2), (1,3) for Player A).	LCD displays "Player A Wins."	Displayed win message.	Pass
F07	Draw Detection	Fill the grid completely without a winning pattern.	LCD displays "Game Over: Draw."	Displayed draw message.	Pass

Hardware Testing

Test case no	**Test Case**	**Input**	**Expected Output**	**Actual Output**	**Pass/Fail**
H02	Button Responsiveness	Rapidly press navigation and placement buttons.	System registers all inputs immediately without any lag.	Buttons responded instantly.	Pass
H03	LED Blinking Behavior	Start the game and observe LED behavior for turns.	Active player's LED blinks; inactive player's LED stays off.	LED blinked correctly.	Pass
H04	Buzzer Distinct Sounds	Press navigation and placement buttons.	Buzzer emits short beeps for navigation and long beep for placement.	Sounds matched actions.	Pass

Integration Testing

Test case no	Test Case	Input	Expected Output	Actual Output	Pass/Fail
I01	Data Communication Accuracy	Send Player A's mark to Player B's module.	Player B's module updates the grid correctly without data loss.	Communication is error-free.	Pass
I02	Game Logic Validation	Simulate multiple win, draw, and invalid move scenarios.	System correctly handles turns, validates moves, and displays results.	Game logic behaved as expected.	Pass

Performance Testing

Test case no	Test Case	Input	Expected Output	Actual Output	Pass/Fail
P01	Input Responsiveness	Rapidly press buttons during gameplay.	All inputs are registered immediately without delay.	registered inputs instantly.	Pass
P02	Stability Under Load	Activate all peripherals (LCD, LED, Buzzer) during gameplay.	System remains stable and responsive; no crashes or freezes.	System remain stable.	Pass

7.4 Sample Detailed Test Case

Test Case: F03 – Navigation Button Functionality

- **Test ID:** F03
- **Objective:** Validate navigation buttons correctly move the cursor across the grid.
- **Inputs:**
    1. Power on the module.

2. Press the navigation button once (binary representation: 0b01 for right, 0b10 for down).
3. Observe LCD cursor movement.

- **Expected Output:** LCD cursor moves to the next grid cell in the direction of navigation.
- **Cursor position update:**
  - Grid (3x3) Index:
    New_Position = (Current_Row + Row_Offset, Current_Col + Col_Offset)
  - Where:
    Row_Offset = ±1 (up/down movement)
    Col_Offset = ±1 (left/right movement)
- **Actual Output:** *(filled during testing)*
- **Pass/Fail:** Cursor should move to the correct position.

Test Case: F06 – Win Detection

- **Test ID:** F06
- **Objective:** Verify the system correctly detects a win condition.
- **Inputs:**
  1. Player A places marks to complete the top row:
     - (1,1) → Press navigation button, then placement button.
     - (1,2) → Press navigation button, then placement button.
     - (1,3) → Press navigation button, then placement button.
  2. Observe LCD display.
- **Expected Output:**
  - LCD displays "Player A Wins."
  - Game stops further input.
  - Player A's buzzer emits a single long beep.
- **Actual Output:** *(Filled during testing)*
- **Pass/Fail Criteria:**
  - **Pass:** LCD displays win message, game halts, buzzer sounds once.

- **Fail:** Incorrect message, game continues, or no buzzer feedback.

7.5 Bit-Level Formula for Key Functionalities

1. **Cursor Movement:**
    - New Position = (Current_Row + Row_Offset, Current_Col + Col_Offset)
    - Where:
        - Row_Offset = ±1 (up/down).
        - Col_Offset = ±1 (left/right).
2. **Win Detection (Row Example):**
    - Binary Representation:
        - Player A's Row 1 = 0b111 (all cells marked by Player A).
        - Row_1 & Player_Mask == Player_Mask
3. **Voltage Regulation Tolerance:**
    - **Min Voltage:** 3.3V - (3.3 * 0.05)
    - **Max Voltage:** 3.3V + (3.3 * 0.05)

7.6 Test Execution Summary

Test case no	Status	Remarks
F01	Pass/Fail	Observations added here
H01	Pass/Fail	Observations added here
I01	Pass/Fail	Observations added here

8 Calculations

Component	Calculation	Value/Formula	Explanation
UART Baud Rate	To calculate the baud rate for UART communication.	f_Clock / (16 * (UCA0BR0 + UCA0BR1))	With f_Clock = 1MHz, the calculated baud rate is 9600, matching the desired baud rate for communication.
Debounce Timer Interval	Timer interval for button debounce.	Timer Interval = 60000 (value in TA0CCR0)	This results in a debounce interval that ensures a consistent signal even if buttons are pressed or released quickly.
Buzzer Sound Frequency	Buzzer frequency used for sound generation.	frequency = 1000Hz (used in buzzer_emit_sound)	Used to emit a tone that indicates a valid move or a change in marker.
Delay for Buzzer Sound	Delay time between buzzer on/off cycle.	delay = 1000000 / frequency / 2	For a frequency of 1000Hz, the duration of each cycle is calculated based on this formula to control buzzer sound length.
Grid Drawing Timer	Time taken for drawing the game grid on the LCD using I2C.	I2C Data Rate = ~100kHz (set by UCB0BR0 = 10)	Determines how quickly grid lines can be drawn on the screen.
LED Blinking Timer	Time duration to keep the LED on/off indicating player turns.	LED Duration = 500ms	The LED blinks to show whose turn it is, controlled by simple timers or delays, e.g., __delay_cycles(500000) to create a 500ms delay for each blink.
Grid Pixel Placement	Pixel placement on the LCD grid for the marker.	pixelX = x * 42 + 14 for horizontal, pixelY = y * 21 + 7 for vertical	Maps logical coordinates to pixel coordinates on the screen. This allows for proper positioning of 'X' or 'O' markers in the game.
Reset Timer	Time taken for resetting the game, clearing grid and resetting hardware components.	resetDelay = 500ms	A delay is added to ensure the reset is smooth and consistent, giving time to reinitialize the display and hardware properly.

9 Setbacks and their solutions

Throughout the development of the Tic Tac Toe game, we encountered several challenges that required creative problem-solving. One of the biggest hurdles was UART lag, which caused delays in communication between the two boards, affecting the game's responsiveness. To address this, I refined UART communication by using interrupt-driven mechanisms for both sending and receiving data. This ensured smoother gameplay with no noticeable lag.

Another issue was the button press lag, which became noticeable when navigating the marker or placing it on the grid. To fix this, I implemented a debouncing timer, which helped eliminate false button presses and made the controls much more reliable.

After implementing the reset functionality, we faced difficulties with starting a new game after a reset. The game variables and states weren't properly resetting, causing the game to start in an inconsistent state. I solved this by refining the reset Game function, ensuring that it cleared all game data, reinitialized the display, and reset the grid, allowing the game to start fresh every time.

Everything worked smoothly during testing on the breadboard, but when it came time to move to the PCB, we faced a number of new issues, such as inconsistent connections and signal interference, which prevented the final product from working as expected. While we were able to get the game up and running on the breadboard prototype, we weren't able to complete the final PCB version. Nonetheless, the experience taught us a lot about debugging and fine-tuning both hardware and software, and it will definitely inform future projects.

10 Conclusion

The Tic Tac Toe game project has been a rewarding and sometimes frustrating experience. While the game worked perfectly on the breadboard, I ran into a number of challenges when transitioning to the final PCB design. Issues like UART lag, button press delays, and problems with restarting the game after a reset became apparent in the final product. Fortunately, through careful troubleshooting and fine-tuning of the code, I was able to overcome these obstacles. I resolved the UART lag, addressed the button press latency with debouncing techniques, and improved the reset functionality by refining the game state management.

Despite not being able to complete the final product due to persistent challenges with the PCB, this project has deepened my understanding of embedded systems and microcontroller programming. It has taught me how delicate the balance between hardware and software is, and how crucial it is to debug and refine code to ensure that everything works seamlessly in the final build. Even though I didn't reach the finish line, the insights I gained from building the prototype and troubleshooting the issues will be invaluable for future projects.

11 References

- **MSP430 Microcontroller Datasheet**
  Texas Instruments. (2024). *MSP430G2553 Datasheet*.
  https://www.ti.com/lit/ds/symlink/msp430g2553.pdf

- **SSD1306 OLED Display Documentation**

  Adafruit Industries. (2024). *Adafruit SSD1306 OLED Display*.

  https://learn.adafruit.com/adafruit-128x128-oled-breakout

- **Code Composer Studio User Guide**

  Texas Instruments. (2024). *Code Composer Studio User's Guide*.

  https://www.ti.com/tool/CCSTUDIO